



6
35
5250 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

Arithmetic for
computer

دخولة العلي



RB Informatics; 05/05/2026

بنیان الحاسوب ا

بسم الله الرحمن الرحيم

مراجعة عناصر الحوسبة الأساسية

المفك decoder : دائرة منطقية تحول مدخلات مكونة من N bit إلى مخرجات تصل إلى 2^N bit مثال:

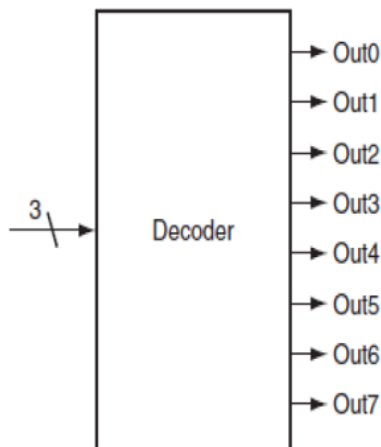
المفك ذو 3 bit ينتج 8 مخرجات محتملة

Review of Basic Computing Elements

2

• Combinational Logic - Decoder

○ N-bits inputs => 2^N bits outputs



a. A 3-bit decoder

Inputs			Outputs							
12	11	10	Out7	Out6	Out5	Out4	Out3	Out2	Out1	Out0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

b. The truth table for a 3-bit decoder

جداول الحقيقة truth table : تستخدم لعرض جميع قيم المداخل الممكنة وبيان قيمة الخرج المقبلة لكل حالة ، مما يساعد على تمثيل وتحليل الدوال المنطقية ويمكن كتابة الدالة المنطقية بطريقتين أساسيتين :

1- (Sum Of Product - SOP) جمع المضارب :

يتم اختيار الصفوف التي تكون فيها قيمة الخرج (1) ثم كتابة حدود مضروبة لكل صف باستخدام (AND) ثم جمع الحدود المضروبة باستخدام (OR)

2- (Poduct Of Sum - POS) ضرب المجاميع :

يتم اختيار الصفوف التي تكون فيها قيمة الخرج (0) ثم كتابة حدود مجموعة لكل صف باستخدام (OR) وضربها باستخدام AND

Truth Tables

4

Inputs			Outputs		
A	B	C	D	E	F
0	0	0	0	0	0
0	0	1	1	0	0
0	1	0	1	0	0
0	1	1	1	1	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	1	0	1

- **Sum of Products**
- $D = (A \cdot B \cdot C) + (A \cdot \overline{B} \cdot C) + (\overline{A} \cdot B \cdot C) + (\overline{A} \cdot \overline{B} \cdot C) + (A \cdot \overline{B} \cdot \overline{C}) + (A \cdot B \cdot \overline{C}) + (\overline{A} \cdot B \cdot \overline{C}) + (\overline{A} \cdot \overline{B} \cdot \overline{C})$

Programmable logic array-PLA : مصفوفة المنطق القابلة للبرمجة : تعد وسيلة فعالة لتنفيذ الدوال المنطقية المركبة تعتمد في تقسيمها على مفهوم sum of products ، وتتكون من مستويين رئيسيين من البوابات المنطقية:

1- مصفوفة بوابات AND :

هي المرحلة الأولى حيث تدخل المدخلات إلى هذه المصفوفة لإنتاج الحدود المضروبة Product terms

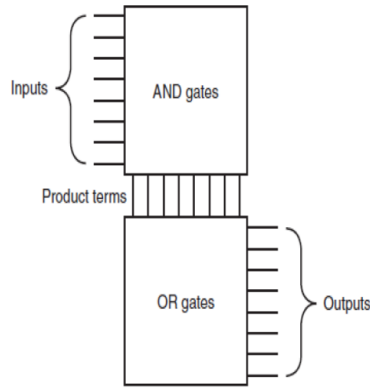
2- مصفوفة بوابات OR :

هي المرحلة الثانية حيث تأخذ الحدود المضروبة الناتجة من المرحلة الأولى وتجمعها معاً (عملية جمع منطقي) لإنتاج المخرجات النهائية التي تسمى الحدود المجموعة Sum term

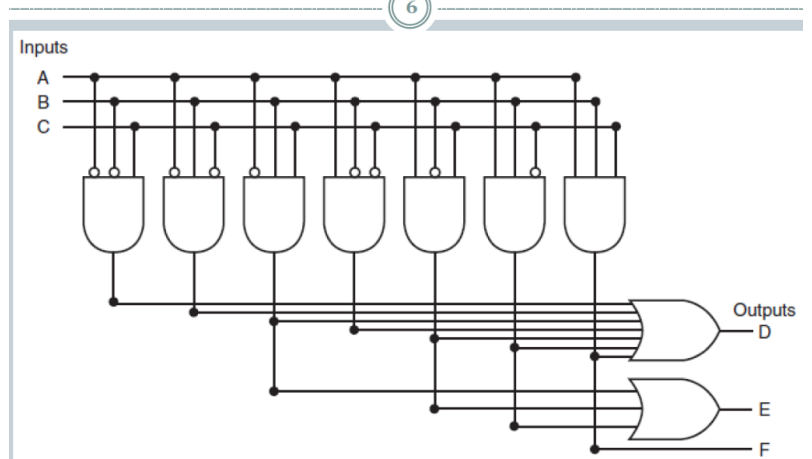
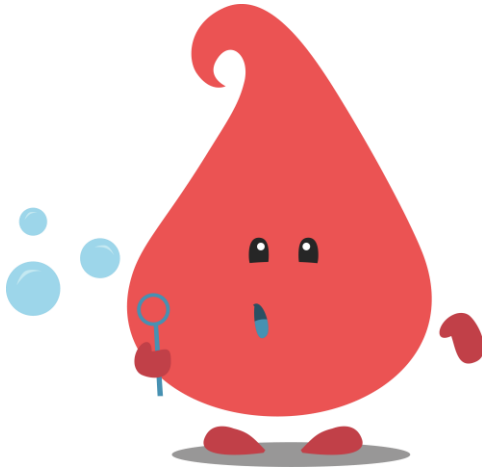
Programmable Logic Array - PLA

- Basic form of a PLA consists of an array of AND gates followed by an array of OR gates.

○ Each entry in the AND gate array is a product term consisting of any number of inputs or inverted inputs. Each entry in the OR gate array is a sum term consisting of any number of these product terms.



Detailed Implementation



ALU Design تصميم وحدة الحساب والمنطق :

1- 1 Bit ALU : تدخل قيمتا a , b إلى الثلاثة مكونات داخل الوحدة في نفس اللحظة :

- بوابة (AND) : تخرج نتيجة $(a \text{ AND } b)$

- بوابة (OR) : تخرج $(a \text{ OR } b)$

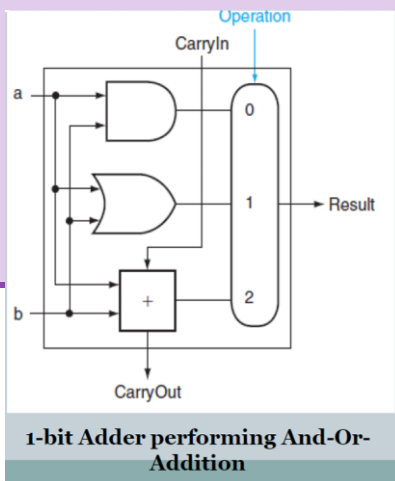
- وحدة الجامع (Adder) : تخرج نتيجة عملية الجمع الحسابي تصل مخرجات هذه العمليات الثلاث إلى المجمع Multiplexer لكنه لا يمرر للخروج النهائي إلا واحدة منها فقط (تسمى Result) بناءً على إشارة التحكم

operation فإذا كانت قيمة ال operation :

- (0): يختار المجمع مخرج بوابة AND

- (1): يختار المجمع مخرج بوابة OR

- (2): يختار المجمع مخرج وحدة الجامع



1-bit Adder performing And-Or-Addition

-الجامع الكامل full Adder : هو الجزء المسؤول

عن العمليات الحسابية يحتاج إلى المدخلين a, b

ومدخل للحمل carry in وينتج مخرجين :

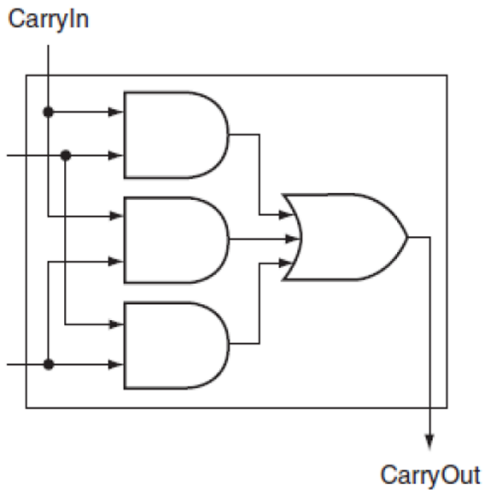
المجموع sum – الحمل الخارج carry out

1-Bit Adder Truth Table

11

Inputs			Outputs		Comments
a	b	CarryIn	CarryOut	Sum	
0	0	0	0	0	$0 + 0 + 0 = 00_{two}$
0	0	1	0	1	$0 + 0 + 1 = 01_{two}$
0	1	0	0	1	$0 + 1 + 0 = 01_{two}$
0	1	1	1	0	$0 + 1 + 1 = 10_{two}$
1	0	0	0	1	$1 + 0 + 0 = 01_{two}$
1	0	1	1	0	$1 + 0 + 1 = 10_{two}$
1	1	0	1	0	$1 + 1 + 0 = 10_{two}$
1	1	1	1	1	$1 + 1 + 1 = 11_{two}$

- **CarryOut** = $(b \cdot \text{CarryIn}) + (a \cdot \text{CarryIn}) + (a \cdot b) + (a \cdot b \cdot \text{CarryIn})$
 $= (b \cdot \text{CarryIn}) + (a \cdot \text{CarryIn}) + (a \cdot b)$
- **Sum** = $(a \cdot \overline{b} \cdot \overline{\text{CarryIn}}) + (a \cdot \overline{b} \cdot \text{CarryIn}) + (a \cdot b \cdot \overline{\text{CarryIn}}) + (a \cdot b \cdot \text{CarryIn})$

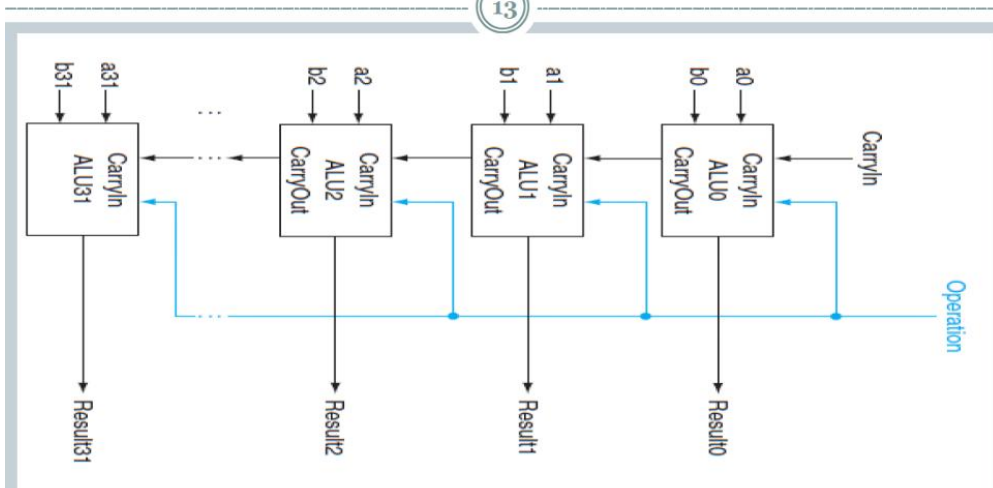


Designing the CarryOut Logic

2- 32-Bit ALU : يتم بناءها عبر ربط 32 وحدة ALU بقياس (1-bit) معاً ، كل وحدة تتعامل مع مدخلين ، ومدخل لقيمة الحامل القادم من الوحدة السابقة carry in وتعطي ناتج البت result والحمل الناتج carry out

32-Bit ALU

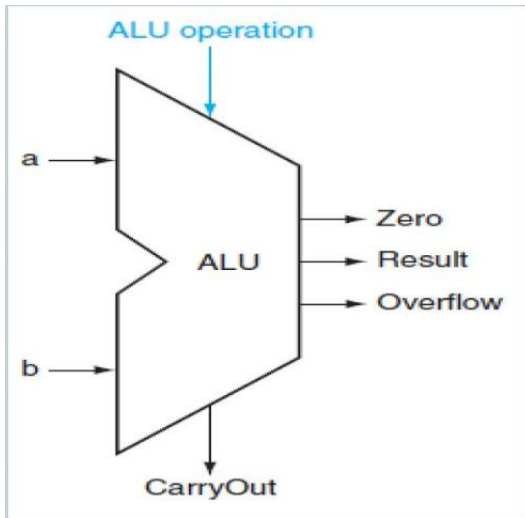
13



- **Faster Addition?** Carry Look-ahead Adder, and Ripple Carry Adder

تسمى هذه الطريقة بـ Ripple Carry Adder أي ينتقل الحمل carry تدريجياً من بت إلى البت الذي يليه ، هذه الطريقة بسيطة لكنها بطيئة عند التعامل مع عدد كبير من البتات لذلك تم تطوير طريقة أسرع تسمى : Carry Look-ahead Adder بدلاً من انتظار انتقال الحمل خطوة خطوة تقوم الدائرة بحساب الحمل المتوقع مسبقاً باستخدام معادلات منطقية، وهذه الطريقة أسرع وأكثر كفاءة وبالتالي تحسين السرعة حيث يتم "جامع استباق الحمل (Carry Look -ahead Adder) " ليكون أسرع من "الجامع المتسلسل (Ripple Carry Adder) " الذي ينتظر انتقال الحمل من بت إلى آخر

رمز ALU في معمارية MIPS :



يستخدم لتمثيل وحدة الحساب والمنطق كاملة كتلة واحدة بدلاً من رسم وحدات 1-Bits بشكل منفصل. تحتوي على مدخل للبيانات a, b وإشارة تحكم operation لتحديد العملية المطلوبة ، تنتج مخرج النتيجة result ، بالإضافة إلى إشارات حالة مثل zero (إذا كانت النتيجة صفر ، تستخدم في تعليمات المقارنة والقفز و الشرطي لمعرفة إذا كانت القيمتان متساويتين أم لا) و overflow و carry out الحمل الناتج.

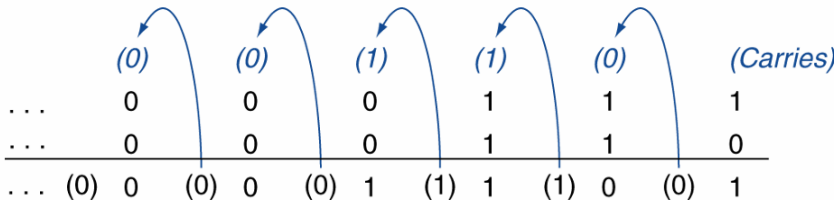
Arithmetic for computers

: Operation on integers

: Addition and subtraction

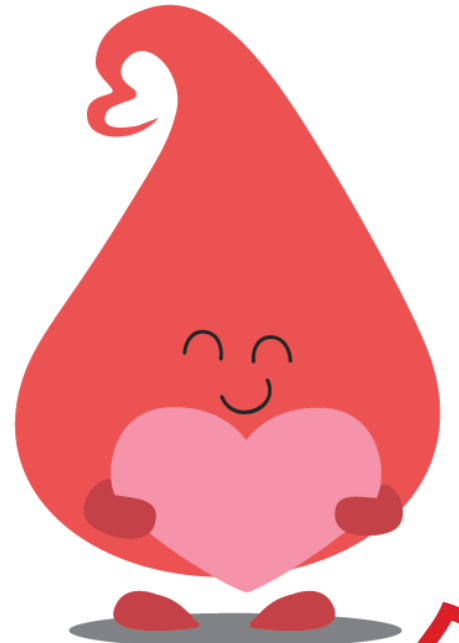
تتم عملية الجمع من خلال انتقال الحمل carry بين البتات المتتالية ، مثال : 7+6

■ Example: 7 + 6



■ Overflow if result out of range

- Adding +ve and -ve operands, no overflow
- Adding two +ve operands
 - Overflow if result sign is 1
- Adding two -ve operands
 - Overflow if result sign is 0



يحدث overflow عندما تتجاوز النتيجة النطاق المسموح للتمثيل، مثل جمع عددين موجبين (بت الإشارة = 0) فتكون النتيجة سالبة (بت الإشارة = 1)

تتم عملية الطرح بتحويل العدد السالب إلى مكمله الثنائي two's complement ثم جمعه العدد الآخر مثال:

- Add negation of second operand

- Example: $7 - 6 = 7 + (-6)$

```
+7: 0000 0000 ... 0000 0111
-6: 1111 1111 ... 1111 1010
+1: 0000 0000 ... 0000 0001
```

- Overflow if result out of range

- Subtracting two +ve or two -ve operands, no overflow
- Subtracting +ve from -ve operand
 - Overflow if result sign is 0
- Subtracting -ve from +ve operand
 - Overflow if result sign is 1



تحدث حال overflow :

عند طرح عدد موجب من عدد سالب وكانت النتيجة موجبة (0)

عند طرح عدد سالب من عدد موجب وكانت النتيجة سالبة (1)

Dealing with overflow : التعامل مع الفيض :

بعض اللغات مثل c تتجاهل الفيض بينما لغات أخرى مثل (Ada-fortran) تتطلب إطلاق استثناء Exeption التعليمات مثل : $subu - addu$ تنفذ بشكل طبيعي دون إطلاق استثناء

بينما تعليمات مثل : $add - sub$ قد تطلق أثناء حدوث حالة overflow، ويتم التعامل معه كالتالي :

1- يتوقع البرنامج عن التنفيذ الطبيعي وينتقل التحكم تلقائياً إلى برمجية خاصة تسمى exception handler (وهو جزء من نظام التشغيل مهمته التعامل مع الأخطاء الغير متوقعة)

2- حفظ عداد البرنامج في سجل EPC : قبل الانتقال إلى معالج الاستثناء ، يقوم النظام بحفظ عنوان التعليمة التي تسببت في حدوث سجل خاص موجود في "عداد البرنامج" pc

3- القفز إلى عنوان المعالج المحدد مسبقاً : ينتقل المعالج للعمل عند عنوان ذاكرة محددة مسبقاً حيث توجد التعليمات الخاصة بكيفية التصرف عند حدوث هذا النوع من الأخطاء (مثل إظهار رسالة خطأ للمستخدم أو محاولة تصحيح النتيجة)

4- استخدام تعليمة mfco للعودة :

بعد انتهاء معالج الاستثناءات من اتخاذ الإجراءات التصحيحية يحتاج النظام للعودة إلى البرنامج الأصلي ، لذلك تستخدم تعليمة mfo (move from coprocessor 0) لاستعادة القيمة المخزنة في السجل Epc بمجرد استرجاع العنوان ، يمكن للمعالج العودة ومواصلة تنفيذ البرنامج من النقطة التي يتوقف عندها بعد معالجة الخطأ

وبالتالي آلية معالج الاستثناء: عند حدوث overflow لا يتم فقط القفز للمعالج مباشرة بل يقوم الجهاز بحفظ عنوان التعليمات التي سببت الخطأ في سجل خاص يسمى (Counter Program Exception EPC) ويتم استخدام تعليمة mfc0 لاستعادة هذه القيمة والعودة للبرنامج بعد تصحيح الخطأ

: Multimedia Arithmetic

تعتمد الرسوميات و الوسائط على :

تمثل البيانات على شكل متجهات vectors :

حيث يتم التعامل مع البيانات على شكل مجموعات صغيرة من القيم مثل 8 أو 16 بت (مثل قيم الألوان أو عينات الصوت ، بدل معالجة قيمة واحدة في كل مرة)

✓ تقنية SIMD(Single-instruction multimedia) :

أي (تعليمية واحدة ، بيانات متعددة) حيث يتم تنفيذ نفس التعليمية على عدة بيانات في نفس الوقت ، مما يسرع عمليات المعالجة بشكل كبير في تطبيقات الصور والفيديو و الصوت

✓ العمليات التشبعية (Saturating operations) :

عند حدوث فيض في القيم، لا تلتف القيمة كما في الحساب العادي بل يتم تثبيت النتيجة عند أقصى قيمة ممكنة (الحد الأعلى او الأدنى الممكن)، مما يمنع حدوث تشوه مفاجئ في الصوت أو الصورة، مثل بقاء قيمة اللون ضمن المجال المسموح (0→255) وتحديد مستوى الصوت عند أقصى قيمة دون تجاوزها

فائدة :

اللاتفاف : هو سلوك يحدث في الحسابات الرقمية عند تجاوز القيمة المجال المسموح للتمثيل ، فبدلاً من إظهار خطأ أو إيقاف العملية ، تعود القيمة إلى الطرف الآخر من المجال

الآلية العمل :عندما يصل العدد إلى أقصى قيمة يمكن تمثيلها في عدد معين من البتات (Bits) ثم يزداد، فإنه يبدأ من أصغر قيمة مجدداً

مثال توضيحي :في نظام مكون من 8 بت يتراوح مدى العداد من 0 إلى 255 ف إذا كانت القيمة الحالية هي 255 وأضفت إليها 1، فإن النتيجة في الحساب العادي هي 256، ولكن بسبب محدودية البتات، ستلتف القيمة وتصبح 0 -المشاكل الناتجة :يُعتبر اللتفاف سلوكاً غير مرغوب فيه في تطبيقات معينة مثل الرسوميات والصوتيات لأنه يعطي نتائج غير منطقية، مثل تحول بقعة شديدة البياض فجأة إلى اللون الأسود أو حدوث ضجيج مفاجئ في الصوت وبالتالي البديل هو العمليات التشبعية لتجنب اللتفاف

باختصار: اللتفاف هو النتيجة الطبيعية إهمال " overflow " في الأنظمة الرقمية، بينما التشبع هو وسيلة عتادية (Hardware) للتعامل معه بشكل يحافظ على منطقية البيانات

: Multiplication

ينفذ المعالج عملية الضرب الثنائية بطريقة الضرب الطويل (long multiplication) التي نستخدمها يدوياً في النظام العشري :



المضروب multiplicand → 1000
المضروب فيه multiplier → 1001

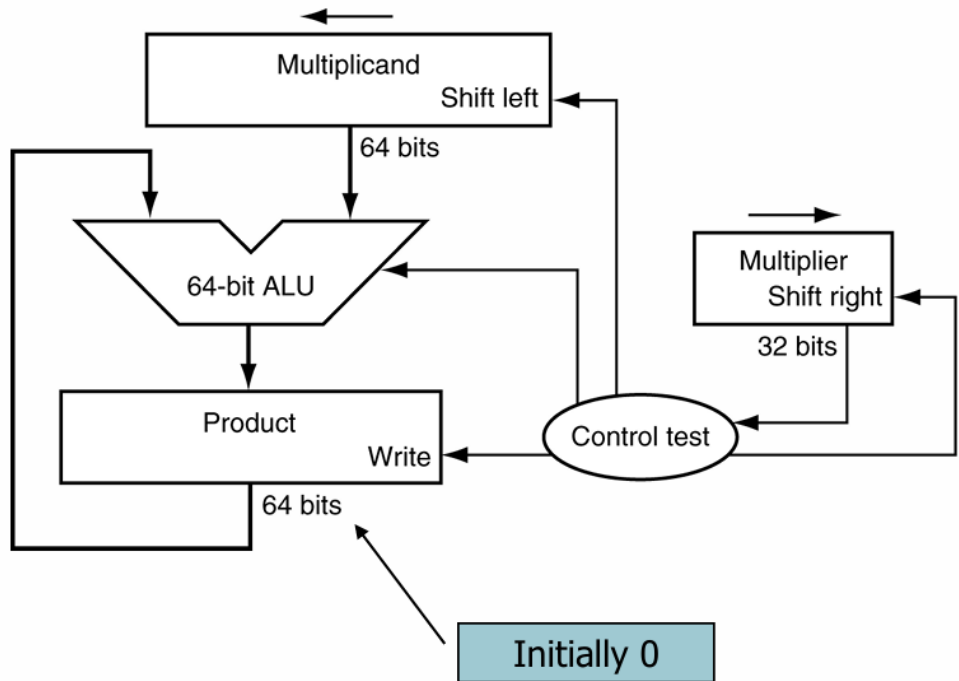
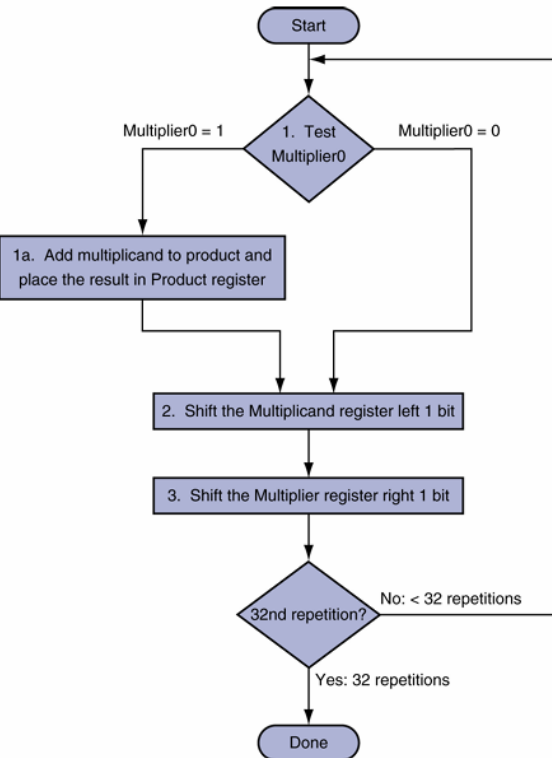
$$\begin{array}{r} 1000 \\ \times 1001 \\ \hline 1000 \\ 0000 \\ 0000 \\ 1000 \\ \hline 1001000 \end{array}$$

الناتج product → 1001000

Length of product is the sum of operand lengths

ملاحظة: أقصى طول للناتج = عدد بتات المضروب + عدد بتات المضروب فيه

: Multiplication Hardware



: Multiplicand Register 1.

يحفظ الرقم المراد ضربه

حجمه 64 بت

كل دورة يقوم ب shift left

الهدف : محاكاة انتقال الرقم يساراً في الضرب الطويل

2. Multiplier register :

يحفظ الرقم الذي نضرب به

حجمه 32 بت

كل دورة ب shift right

السبب: حتى نفحص البت التالي كل مرة (0 أو 1)

3. Product register :

يحفظ الناتج الجزئي و النهائي

حجمه 64 بت يبدأ بقيمة 0

4. ALU (Arithmetic logic unit) :

وظيفتها : تجمع multiplicand مع product عندما تكون قيمة البت الحالية = 1

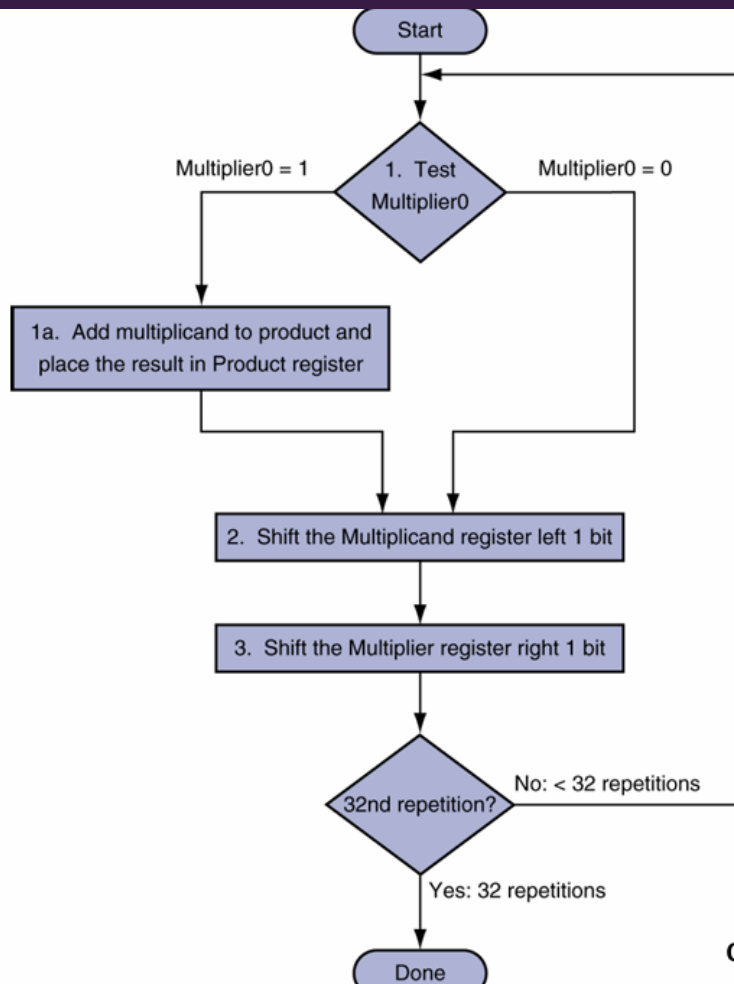
5. Control test وحدة التحكم :

تفحص أقل بت في multiplier

إذا كان = 1 نجمع

إذا كان = 0 لا نجمع ... ثم نتحكم بعمليات الإزاحة

خطوات الخوارزمية Flow chart



Cha

الخوارزمية تتكرر 32 مرة (بعدد البتات):الخطوة الأولى :

فحص أقل بت من Multiplier أي : (LSB) إذا كانت قيمته = 1
نفذ $product = product + Multiplicand$ إذا كانت قيمته = 0 ، لا يحدث جمع

الخطوة الثانية :

إزاحة ال multiplicand خانة لليسار
الهدف: تجهيز المضروب للمكان التالي في الضرب الطويل

الخطوة الثالثة :

إزاحة ال multiplier خانة لليمين
الهدف: للحصول على البت التالي لفحصه (1 // 0)

الخطوة الرابعة :

التكرار ، نعيد الخطوات السابقة حتى ننتهي من جميع البتات

مثال:

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	0011 ¹	0000 0010	0000 0000
1	1a: $1 \Rightarrow Prod = Prod + Mcand$	0011	0000 0010	0000 0010
	2: Shift left Multiplicand	0011	0000 0100	0000 0010
	3: Shift right Multiplier	000 ¹	0000 0100	0000 0010
2	1a: $1 \Rightarrow Prod = Prod + Mcand$	0001	0000 0100	0000 0110
	2: Shift left Multiplicand	0001	0000 1000	0000 0110
	3: Shift right Multiplier	000 ⁰	0000 1000	0000 0110
3	1: $0 \Rightarrow$ No operation	0000	0000 1000	0000 0110
	2: Shift left Multiplicand	0000	0001 0000	0000 0110
	3: Shift right Multiplier	000 ⁰	0001 0000	0000 0110
4	1: $0 \Rightarrow$ No operation	0000	0001 0000	0000 0110
	2: Shift left Multiplicand	0000	0010 0000	0000 0110
	3: Shift right Multiplier	000 ⁰	0010 0000	0000 0110

شرح المثال:

القيم الابتدائية :

Multiplicand = 0010

Multiplier = 0011

Product = 0000

أبو حامد الغزالي " :العلم بلا عمل جنون، والعمل بغير علم لا يكون".

-1 الخطوة (1):

فحص أقل بت من multiplier البت = 1 إذاً :

$$\text{Product} = \text{product} + \text{multiplicand}$$

$$= 0000 + 0010$$

$$\text{Product} = 0010$$

shift left for multiplicand

$$\text{multiplicand} = 0100$$

shift right for multiplier

$$\text{multiplier} = 0001$$

-2 الخطوة (2):

نحسّل بت = 1 -> إذاً نجمع

$$\text{Product} = 0010 + 0100 = 0110$$

$$\text{Multiplicand} = 1000$$

$$\text{Multiplier} = 0000$$

-3 الخطوة (3):

نحسّل أقل لت = 0 -> إذاً لا نجمع

$$\text{Multiplicand} = 0001\ 0000$$

$$\text{Multiplier} = 0000$$

-4 الخطوة (4):

كل البتات صفر إذاً لا يوجد جمع

بعد الانتهاء من جميع الدورات ، ينتج: $\text{product} = 0110$

: Optimized multiplier

في دائرة الضرب السابقة هناك عدة مشاكل منها :

1- في كل دورة يوجد عمليتي shift

2- سجلات أكبر

3- هاردوير أكثر ، وبالتالي استهلاك طاقة أكبر

في التصميم الجديد :

Multiplicand يبقى 32 bit

product يبقى 64 bit

يتم دمج multiplier داخل product register في الجزء السفلي

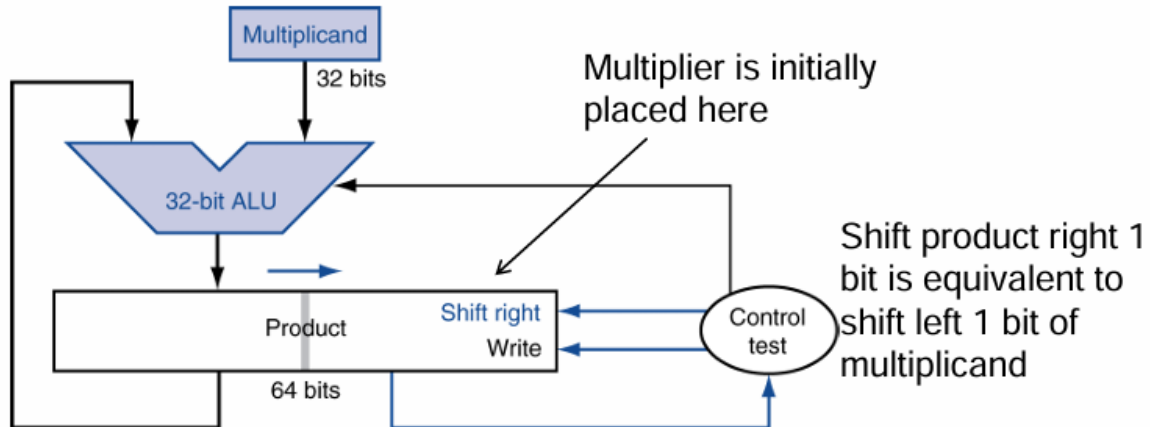
وتحدث عملية إزاحة واحدة لليمين في product register



مثال: multiplicand = 1000

Multiplier = 1001

فيكون product register = 0000 1001



آلية العمل:

1. الخطوة 1:

يتم فحص أقل بت من product Register إذا كان يساوي الواحد تتم عملية الجمع

2. الخطوة 2:

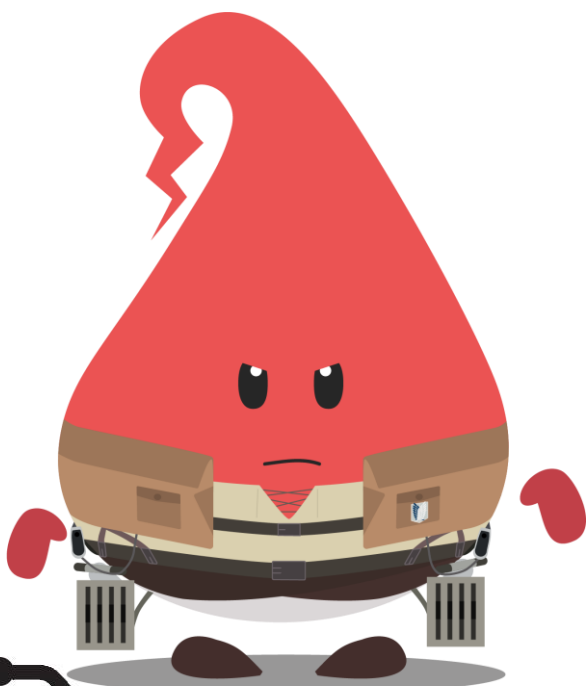
تنفيذ عملية الجمع:

يتم جمع ال-Multiplicand مع الجزء العلوي من ال-product Register

3. الخطوة 3:

Shift Right :

يتم عمل Shift Right لل-product كامل.



Abraham Lincoln:
 "Those who deny
 freedom to others
 deserve it not for
 themselves"

مثال:

Multiplicand = 1000
Multiplier = 1001
Product = 0000 1001

الدورة الأولى :

- فحص لأقل بت في product البت = 1 (تنفذ عملية الجمع)
- جمع $0000 + 1000 = 1000$
- فيصبح : product = 1000 1001
- Shift Right : product = 0100 0100

الدورة الثانية :

- أقل بت = 0 (لا تنفذ عملية الجمع)
- Shift Right : product = 0010 0010

الدورة الثالثة :

- أقل بت = 0 (لا يوجد جمع)
- Shift Right : product = 0001 0001

الدورة الرابعة :

- أقل بت = 1 (يوجد جمع)
- $Product = 0001 + 1000 = 1001$
- Shift Right : product = 0100 1000
- الناتج النهائي : 1001000

ميزة Optimized Multiplier :

تقليل الهاردوير تقليل عدد عمليات shift زيادة السرعة من خلال تنفيذ عملية الجمع مع الإزاحة على التوازي (داخل نفس الدورة الزمنية)

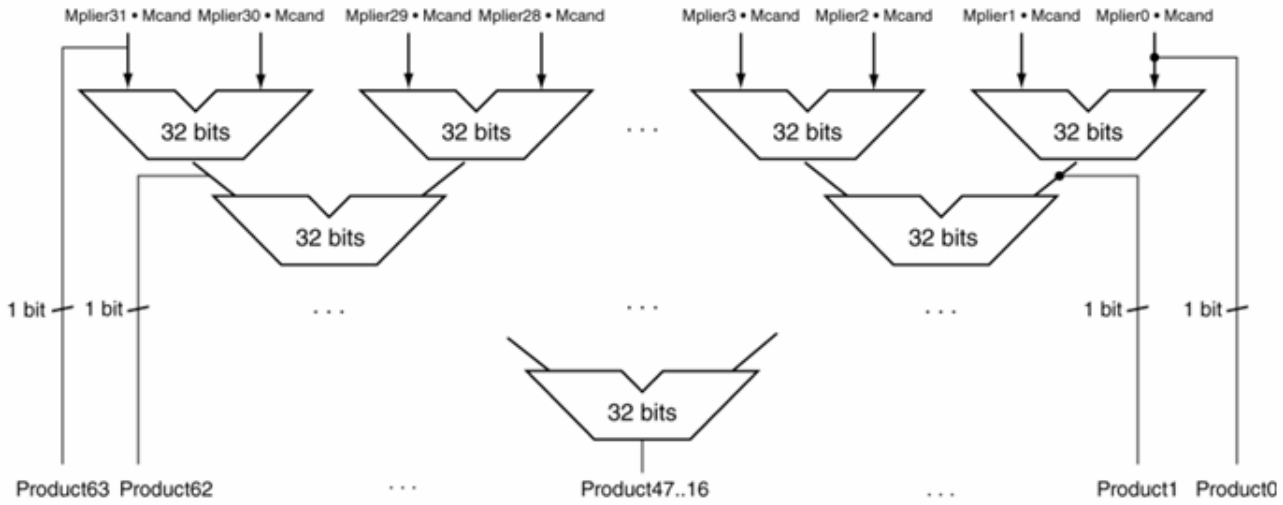
: One Cycle prepartial-product addition

أي كل دورة clock تقوم بـ:

- 1- فحص البت
- 2- تنفيذ الجمع
- 3- تنفيذ shift
- مرة واحدة فقط



Faster Multiplier المضاعف الأسرع :



تعتمد على استخدام عدة دوائر (Multiple address) تعمل بالتوازي ، بدل استخدام جامع واحد وتنفيذ عمليات الجمع بالتتابع

يتم توليد جميع النواتج الجزئية partial product في نفس الوقت ، ثم جمعها معاً مما يقلل عدد الدورات اللازمة لتنفيذ عملية الضرب ويزيد السرعة بشكل أكبر
لكن هذه الطريقة تحتاج إلى هاردوير أكثر ، عدد أكبر من الـ Address ، استهلاك طاقة ومستحقة أكبر لذلك يوجد توازن بين الكلفة cost والأداء performance
كما يمكن تطبيق تقنية التوارد pipeline لتنفيذ عدة عمليات ضرب بالتوازي داخل المعالج



Do it for the younger
version of you ... who
could not

MIPS Multiplication (الضرب في MIPS)

عند ضرب عددين لكل منهما 32 بت قد تكون النتيجة حتى 64 بت لذلك يستخدم MIPS سجلين لتخزين الناتج:

1. HI Register :

يخزن أعلى 32 bit من الناتج (Most significant bits)

2. LO Register :

يخزن أقل 32 bit من الناتج (Least significant bits)

تعليمات الضرب:

1. mult rs , rt :

تستخدم لضرب عددين موقعين signed

2. multu rs , rt :

تشبه mult لكن للأعداد الغير موقعة unsigned

-وفي كلا التعليمتين يتم تخزين الناتج 64 bit داخل السجلين LO/HI

تعليمات نقل القيمة :

1. mfhi rd :

تنقل قيمة السجل HI إلى السجل rd

2. mflo rd :

تنقل قيمة السجل LO إلى السجل rd

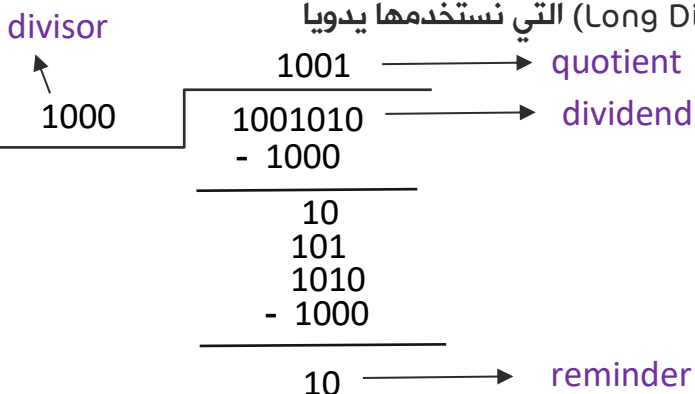
ملاحظات: إذا كانت قيمة $HI \neq 0$ فهذا يعني أن الناتج أكبر من 32 بت، أي حدث overflow

تعليمات : mul rd ,rs ,rt

تضرب عددين وتخزن فقط أقل 32 bit من الناتج في السجل rd بينما يتم تجاهل البتات العليا

: Division

عملية القسمة في الحاسوب تشبه القسمة الطويلة (Long Division) التي نستخدمها يدوياً



يجب التحقق أن القاسم ليس صفراً

خطوات القسمة Long Division App Approach

تنفذ القسمة على كل بت بالتابع

الحالة الأولى : إذا كان $Divisor \leq Dividend \text{ bits}$

أي المقسوم عليه أصغر من أو يساوي القيمة الحالية للمقسوم ونقوم بـ:

1. وضع (1) في خارج القسمة Quotient

2. طرح المقسوم عليه من المقسوم

الحالة الثانية : $Divisor > Dividend$

نقوم بـ:

1. وضع (0) في خارج القسمة

2. إنزل البت التالي من المقسوم

Restoring Division Algorithm خوارزمية القسمة الاسترجاعية

هي خوارزمية القسمة المستخدمة غالباً في العتاد Hardware آلية العمل:

- نطرح أولاً:

$$Reminder = Reminder - Divisor$$

- ثم نتحقق، إذا كان:

$$Reminder \geq 0$$

يعني الطرح صحيح $\Leftarrow$ نضيف 1 إلى خارج القسمة $\Leftarrow$ نكمل

- إذا :

$$Reminder < 0$$

نقوم باستعادة القيمة

$$Reminder = Reminder + Divisor$$

(لذلك اسمها restoring) ثم نضيف 0 إلى خارج القسمة ((سيتم شرحها بالتفصيل))

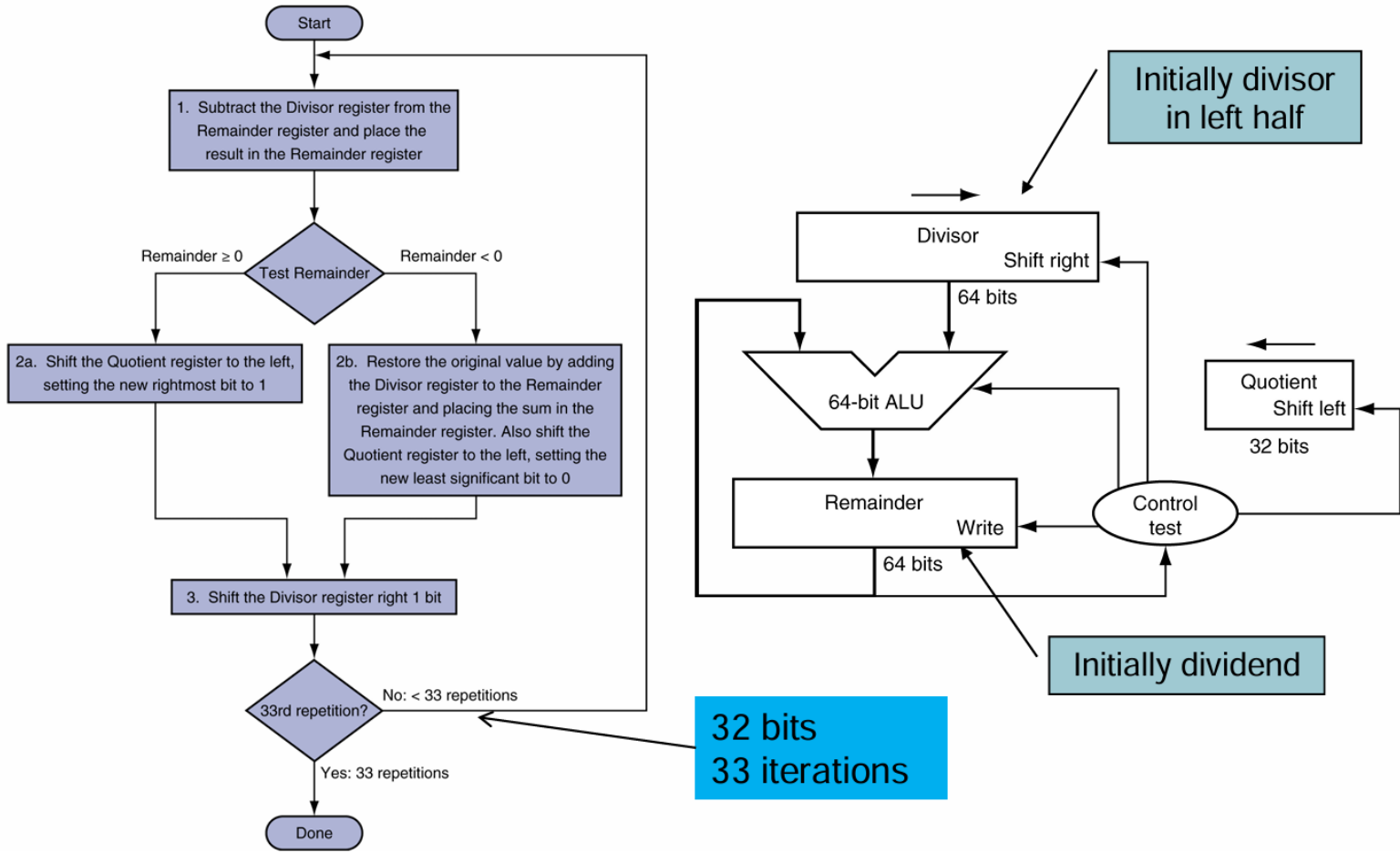
القسمة على الأعداد السالبة Sign division

تنفذ القسمة على القيم المطلقة ثم نعدل الإشارة ، مثال :

$$-8 \div 2 \rightarrow 8 \div 2 = 4$$

أي -4 (نرجع الإشارة)

Division Hardware



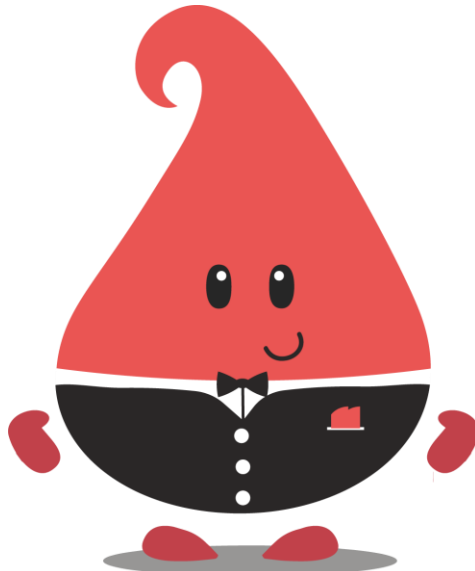
يوجد ثلاث سجلات رئيسية:

ال divisor register : يخزن المقسوم عليه، حجمه 64 bit ، يقوم ب shift right

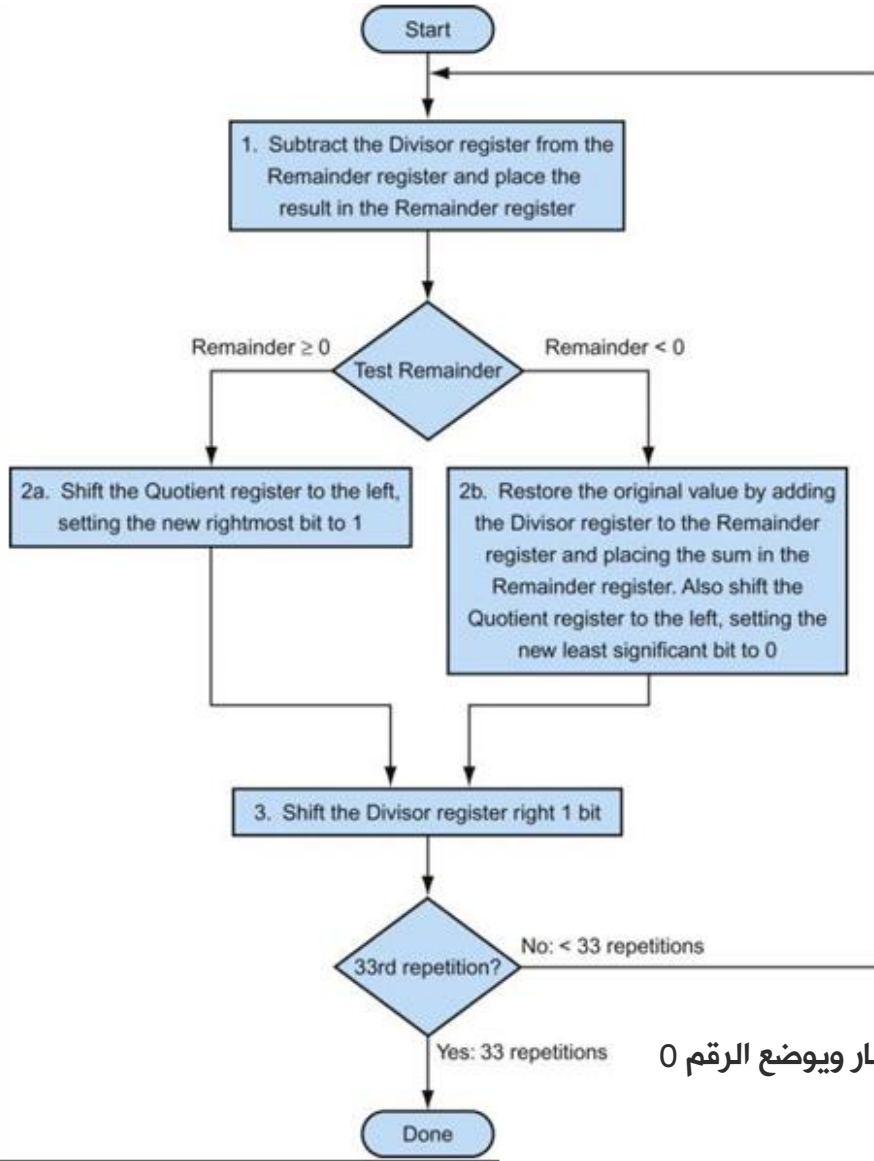
ال remainder register : يخزن الباقي المؤقت أثناء العمليات، حجمه 64 bit يتم تهيئته بالمقسوم dividend

ال quotient register : يخزن خارج القسمة تدريجيا

تعمل هذه السجلات مع ال ALU والتي تنفذ عمليتي الطرح، و الجمع(عند الاستعادة)



خطوات تنفيذ Restoring Division في العتاد



تتكون هذه العملية من 33 تكراراً (الأرقام بطول 32 بت) وتتضمن الخطوات التالية:

الخطوة 1:

الطرح :

Remainder - Remainder - Division

الخطوة 2:

اختبار الباقي :

إذا $Remainder \geq 0$

نقوم بـ: إزاحة خارج القسمة (Quotient) لليسار ثم نضع $Q_0 = 1$ (البت الأقل أهمية)

Quotient ←

إذا $Remainder < 0$

فهذا يعني أن القاسم الأكبر من الجزء المأخوذ من المقسوم

هنا يتم القيام بالخطوتين :

الاسترجاع: يتم إضافة القاسم لسجل الباقي

لاستعادة قيمته السابقة قبل الطرح

$Remainder = Remainder + Divisor$

$Q_0 = 0$ يتم إزاحة سجل القسمة Quotient لليسار ويوضع الرقم 0

في الخانة الأقل أهمية

الخطوة 3:

إزاحة سجل القاسم Divisor إلى اليمين بمقدار بت واحد للمقارنة مع الجزء التالي من المقسوم في الدورة القادمة

الخطوة 4:

يتم التحقق هل تم إكمال 33 تكراراً؟ إذا لم يكن كذلك تعود الخوارزمية للخطوة الأولى وعند الانتهاء يكون خارج القسمة

في سجل Quotient والباقي النهائي في سجل Remainder



مثال:

Iteration	Step	Quotient	Divisor	Remainder
0	Initial values	0000	0010 0000	0000 0111
1	1: Rem = Rem – Div	0000	0010 0000	①110 0111
	2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0	0000	0010 0000	0000 0111
	3: Shift Div right	0000	0001 0000	0000 0111
2	1: Rem = Rem – Div	0000	0001 0000	①111 0111
	2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0	0000	0001 0000	0000 0111
	3: Shift Div right	0000	0000 1000	0000 0111
3	1: Rem = Rem – Div	0000	0000 1000	①111 1111
	2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0	0000	0000 1000	0000 0111
	3: Shift Div right	0000	0000 0100	0000 0111
4	1: Rem = Rem – Div	0000	0000 0100	①000 0011
	2a: Rem $\geq$ 0 $\Rightarrow$ sll Q, Q0 = 1	0001	0000 0100	0000 0011
	3: Shift Div right	0001	0000 0010	0000 0011
5	1: Rem = Rem – Div	0001	0000 0010	①000 0001
	2a: Rem $\geq$ 0 $\Rightarrow$ sll Q, Q0 = 1	0011	0000 0010	0000 0001
	3: Shift Div right	0011	0000 0001	0000 0001

شرح المثال:

القيم الابتدائية :

Quotient = 0000
 Divisor = 0010 0000
 Remainder = 0000 0111

1. الدورة الأولى :

الدورة الأولى :

قيمة سالبة $r = r - d \rightarrow r = 0000 0111 - 00100000$, r

إذا نقوم باستعادة القيمة

Remainder = 0000 0111

ثم إزاحة Quotient : Q=0000 ووضع (0) في البت أقل أهمية :

أي يبقى يساوي $Q = 0000$

ثم إزاحة المقسوم عليه لليمين :

Divisor = 0001 0000

الدورة الثانية :

نطرح

Remainder = 0000 0111 - 00010000

Reminder = 0000 0111

إزالة Quotient ووضع $Q=0000 \Leftarrow Q_0 = 1$

Divisor = 0000 1000 : Divisor الإزاحة

الدورة الثالثة:

الطرح أيضاً سالب

Divisor = 0000 0100 : Divisor الإزاحة

الدورة الرابعة:

Reminder = 0000 0111 - 0000 0100 = 0000 0011 : **الطرح**

النتاج موجب

Quotient = 0001 $\Leftarrow Q_0 = 1$ نضع

Divisor = 0000 0010 : Divisor إزاحة

الدورة الخامسة:

نظرح : 0000 0001 = Remainder

((موجب))

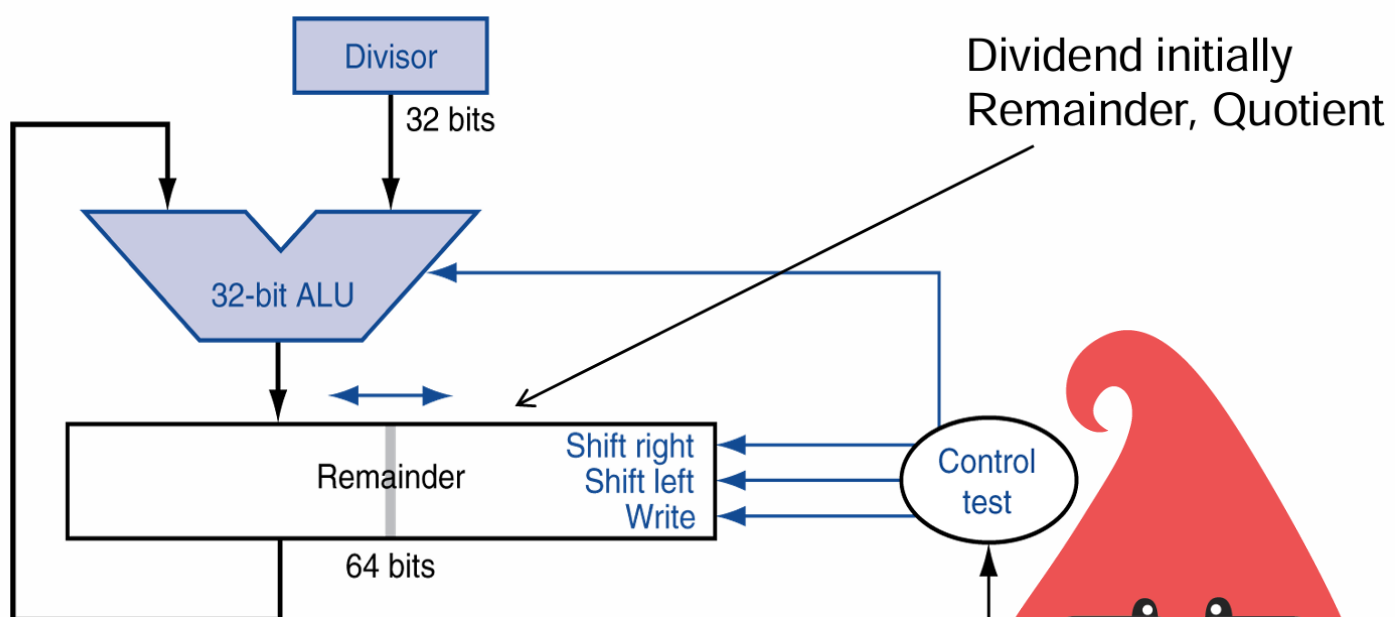
نضع $Q_0 = 1$ بعد الإزاحة : Quotient = 0011

Divisor = 0000 0001 : Divisor الإزاحة

Quotient = 0011 : الناتج

Reminder =1 : الباقي

Optimized Dividers





عبارة عن تصميم أمثر كفاءة ، يهدف لتحسين أداء عملية القسمة ، وتقليل عدد العتاد المستخدم Hardware

حيث تعتمد هذه البنية على تنفيذ عملية طرح جزئي Partial Reminder Subtraction في كل دورة clock حيث يتم استخدام :

1- ALU

2- Reminder Register

3- Control unit

لتنفيذ خطوات القسمة بشكل متكرر

- كما يشبه تصميم هذا القاسم وحدة الضرب multiplier لذلك يمكن استخدام نفس مكونات الهاردوير تقريباً في العمليتين مما يقلل من تعقيد الدارة وكلفتها
 - رغم أن هذا التقسيم أكثر كفاءة من الخوارزمية التقليدية إلا أن عملية القسمة هي تزال أبطأ من عملية الضرب لأن كل خطوة تعتمد على نتيجة الخطوة السابقة مما يحد من إمكانية تنفيذ العمليات بالتوازي
- parallel execution لذلك تم تطوير خوارزمية أسرع مثل : SRTDivision والتي تستطيع توليد عدة bits من خارج القسمة في خطوة واحدة مما يؤدي إلى تحسين سرعة التنفيذ .

MIPS Division

في معمارية MIPS يتم تخزين ناتج عملية القسمة باستخدام سجلين خاصين هما :

1. HI : يخزن الباقي reminder في 32 bit

2. LO : يخزن خارج القسمة Quotient في 32 bit

تعليمات القسمة في MIPS :

1. div rs , rt

تستخدم لإجراء القسمة الموقعة signed

حيث :

rs : يمثل المقسوم dividend

rt : يمثل المقسوم divisor

2. divu rs , rt

تستخدم لإجراء القسمة الغير موقعة unsigned

ملاحظة هامة: معالج MIPS لا يقوم تلقائياً بفحص overflow أو القسمة على صفر divide by zero أي أنه لا يعطي خطأ عند حدوث هذه الحالات لذلك يجب على البرنامج software تنفيذ الفحص يدوياً قبل إجراء القسمة إذا لزم الأمر

ملاحظة كمان:

rd mfhi تستخدم لنقل الباقي من HI إلى rd
rd mflo تستخدم لنقل خارج القسمة من LO إلى rd

مقدمة

سنستخدم طريقة ال Floating_Pionت لتمثيل الأعداد الصحيحة (Non-integral numbers) بما في ذلك الأعداد الكسرية الصغيرة جداً أو الكبيرة جداً (مثال: -2.34×10^{56})

التمثيل المعياري (Normalization):

يكون العدد الثنائي معيارياً عندما يكتب بالصيغة $\pm 10xxxxxx_2 \times 2^{yyyy}$
بينما الأعداد التي لا تبدأ بالرقم 1 قبل الفاصلة الثنائية مثل (0.002×10^{-4}) تعتبر غير معيارية (non_normalized).

■ ملاحظة: الفاصلة العائمة (Floating Point) هي نفس الأعداد من نوع float , double بلغة C

معيّار الفاصلة العائمة IEEE Std 754 1985 معرفّة ب

تم تطوير هذا المعيار لحل الاختلافات بين الأنظمة المختلفة في كيفية تمثيل الكسور، مما ساعد على تحسين إمكانية تشغيل البرمجيات العلمية ونقلها بين الأجهزة، وهذا المعيار هو حالياً مطبق عالمياً بشكل تقريبي (جزئي)

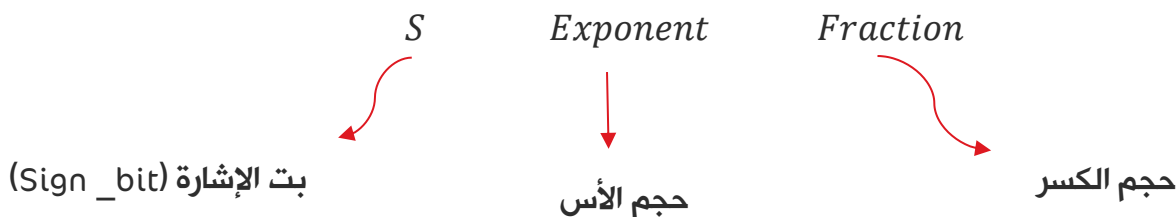
كما أن هذا المعيار يوفر تمثيلين مختلفين رئيسيين:

1. الدقة الأحادية (Single Precision binary) 32_bit
2. الدقة الثنائية (Double Precision binary) 64_bit

■ صيغة الفاصلة العائمة IEEE Floating Point Format
تقسّم هذه الصيغة سلسلة البتات إلى ثلاثة أجزاء مميزة لتمثيل القيمة وباستخدام المعادلة التالية:

$$x = (-1)^S \times (1 + Fraction) \times 2^{(Exponent - Bias)}$$

ويقسّم حجم البتات حسب نوع تمثيلها إلى الصيغة التالية:



Single Precision (32-bit)	1-bit	8-bit	23-bit	Bias (الانحياز) = 127
Double Precision (64-bit)	1-bit	11-bit	52-bit	Bias (الانحياز) = 1023

* شرح كل قسم على حدة :

1. S : بت الإشارة Signal bit (يكون دائماً 1-bit)

إذا كانت البت = 0 تكون الإشارة موجبة.

إذا كانت البت = 1 تكون الإشارة سالبة.

2. الأس المنحاز (Biased Exponent) :

يستخدم الأس تمثيلاً فائضاً 1 منحازاً عبر إضافة قيمة الانحياز "Bias" إلى القيمة الفعلية لضمان بقاء الحقل

مخزن من دون إشارة (سالبة) (unsigned) مما يسهل عملية المقارنة على مستوى الـ Hardware .

3. البت المخفي (Hidden bit) :

يستخدم لزيادة دقة التمثيل إلى أقصى حد، وهو عبارة عن 1. تضاف إلى الأرقام المخزنة في الـ Fraction

ويصبح: اسم العدد الناتج significand جزء المعيارى حيث مجاله $1.0 \leq |\text{significand}| \leq 2.0$ ويتم حذف 1.

للـ significand من حقل الكسر (Fraction) الفعلي المخزن.

نطاق 1 مجال الدقة الأحادية (Single Precision Range):

ملاحظة: الـ Exponents

$$(11111111)_2 \rightarrow (255)_{10} , (00000000)_2 \rightarrow (0)_{10}$$

هما أرقام محجوزة في الذاكرة (Reserved)

* إذاً الآن سنقوم بحساب قيمة أصغر و أكبر عدد في مجال (Single Precision)

$$(00000001)_2 \rightarrow (1)_{10} \quad \text{أصغر رقم للـ Exponent هو}$$

أولاً نحسب القيمة الفعلية للأس Exponent (actual Exponent) كالتالي:

$$\text{actual Exponent} = -\text{Bias}$$

$$\text{actual Exponent} \rightarrow 1 - 127 = -126 \quad \text{Single_Precision} \quad \text{وكما نعلمه}$$

23-bit

بينما الـ Fraction = 000...00 بالتالي $\text{significand} = 1.0$ أي يصبح لدينا

$$\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$$

Binary

Decimal

$$(11111110)_2 \rightarrow (254)_{10}$$

2. أكبر رقم لل Exponent هو

نحسب ال actual Exponent بنفس الطريقة السابقة $254 - 127 = +127$

بينما ال $Fraction = 11111 \dots 11$ بالتالي $signficand = 2.0$ ومنه نحصل على

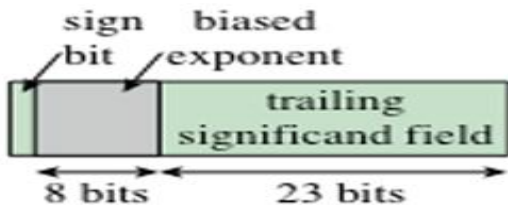
$$\pm 2.0 \times 2^{+127} \approx \pm 3.4 \times 10^{+38}$$

مما سبق نستنتج أن مجال ال Single Precision هو

$$\pm 1.2 \times 10^{-38} \approx \pm 3.4 \times 10^{+38}$$

معلومة إضافية : يتم تقسيم بتات نوع Binary 128 (128 – bit) كالتالي:

Sign bit	Biased Exponent	Fraction
1-bit	15-bit	112-bit



(a) binary32 format



(b) binary64 format



(c) binary128 format

Always remember :

الإمام الشافعي: "العلم ما نَفَعُ،

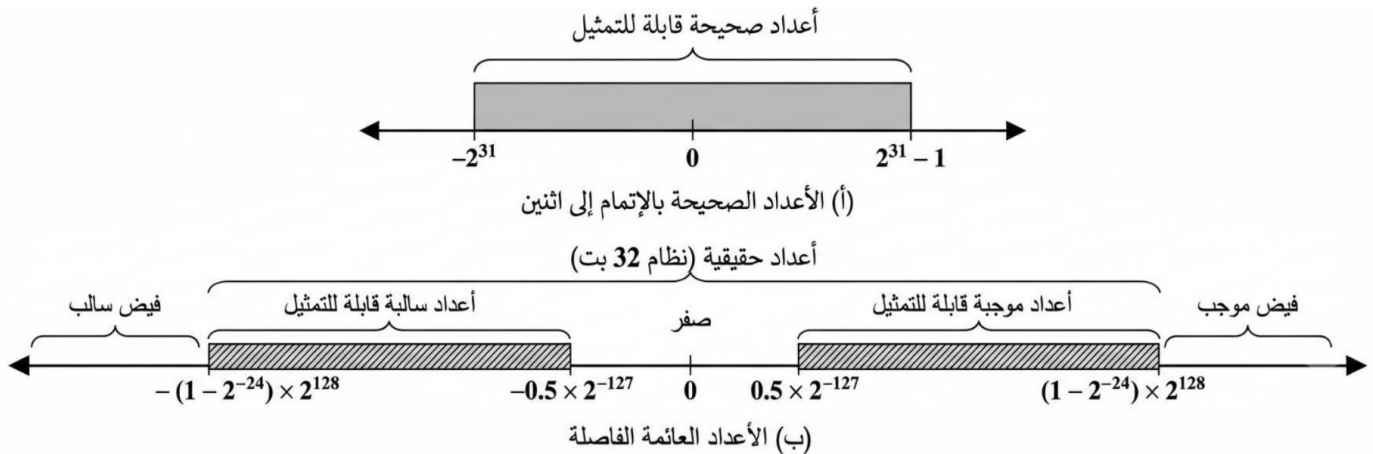
ليس العلم ما حُفِظَ".

Table 10.3

IEEE 754

Format
Parameters

Parameter	Format		
	binary32	binary64	binary128
Storage width (bits)	32	64	128
Exponent width (bits)	8	11	15
Exponent bias	127	1023	16383
Maximum exponent	127	1023	16383
Minimum exponent	-126	-1022	-16382
Approx normal number range (base 10)	$10^{-38}, 10^{+38}$	$10^{-308}, 10^{+308}$	$10^{-4932}, 10^{+4932}$
Trailing significand width (bits)*	23	52	112
Number of exponents	254	2046	32766
Number of fractions	2_{23}	2_{52}	2_{112}
Number of values	$1.98 \times 2_{31}$	$1.99 \times 2_{63}$	$1.99 \times 2_{128}$
Smallest positive normal number	2_{-126}	2_{-1022}	2_{-16362}
Largest positive normal number	$2_{128} - 2_{104}$	$2_{1024} - 2_{971}$	$2_{16384} - 2_{16271}$
Smallest subnormal magnitude	2_{-149}	2_{-1074}	2_{-16494}



نطاق الدقة المزدوجة (Double precision Range):

أيضاً القيمتين 00000000 و 11111111 هما قيمتان محجوزتان (Reserved) في الذاكرة
*سنقوم بحساب مجال Double precision من خلال حساب أصغر قيمة وأكبر قيمة:

1. أصغر قيمة : هي ذات الأس Exponent

$$(000000000001)_2 \rightarrow (1)_{10}$$

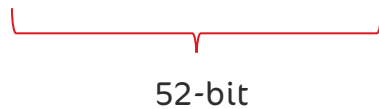
11-bits

نحسب ال actual Exponent والتي تساوي Biased Exponent- 1023 وهنا تساوي $1 - 1023 = -1022$ وبالتالي $Fraction = 000 \dots 00$ ومنه القيمة النهائية تساوي

$$\pm 1.0 \times 2^{-1022} \approx \pm 2.2 \times 10^{-308}$$

52-bit

2. أكبر قيمة ذات الأس $(1111111110)_2 \rightarrow (2046)_{10}$ ومنه نحسب ال actual Exponent والتي تساوي $2046 - 1023 = +1023$ وبالتالي $Fraction = 11111 \dots 11$



وبالتالي $signficand = 2.0$ ومنه القيمة النهائية تساوي $\pm 2.0 \times 2^{+1023} \approx \pm 1.8 \times 10^{+308}$ و مما سبق نستنتج أن مجال ال Double precision هو من

$$\pm 2.2 \times 10^{-308} \approx \pm 1.8 \times 10^{+308}$$

أمثلة عن الفاصلة العائمة Floating Point Example

مثال 1: مثل الرقم -0.75 بال Floating- Point حسب المعادلة

$$x = (-1)^S \times (1 + Fraction) \times 2^{(Exponent - Bias)}$$

$$-0.75 = (-1)^1 \times 1.1_2 \times 2^{-1}$$

$$S = 1, Fraction = 1000 \dots 00_2, Exponent = -1 + Bias$$

$$Single = -1 + 127 = (126)_{10} \quad (01111110)_2$$

$$Double = -1 + 1023 = (1022)_{10} \quad (0111111110)_2$$

Single Precision

1	01111110	100...00
1-bit	8-bit	23-bit

Double precision

1	0111111110	100...00
1-bit	11-bit	52-bit

مثال 2: ماهو الرقم الممثل بال Single Precision (نريد النتيجة بال decimal)

1	10000001	0100 ... 00
---	----------	-------------

Sign – bit = 1 ومنه العدد سالب	Exponent 10000001 ₂ 129 ₁₀	Fraction
-----------------------------------	---	----------

حسب المعادلة

$$x = (-1)^1 \times (1 + 01_2) \times 2^{(129-127)} = (-1) \times 1.25 \times 2^2 = -5.0$$

الأعداد عبر المعيارية (Denormal Numbers)

هي عبارة عن الحالة التالية : يكون ال Exponent عبارة عن أصفار 0=Hidden bit 000...00

$$x = (-1)^S \times (0 + Fraction) \times 2^{-Bais}$$

ملاحظة: في هذه الحالة يكون الكسر غير صفري

فيكون العدد الناتج أصغر من الأعداد الطبيعية وبما أن الكسر غير الصفري والبت المخفي =0 هذا يسهل حالة

(gradual underflow) التدفق السفلي التدريج التي تجنب الهبوط المفاجئ إلى الصفر

ماذا لو كانت قيمة ال Fraction أصفاراً؟ 00...00 Fraction =

$$x = (-1)^S \times (0 + 0) \times 2^{-Bais} = \pm 0.0$$

تمثيلين لل 0.0

الانهاية و NaN

NaN : Not a Number حيث

في حالة الانهاية يكون Exponent = 111...11 , Fraction = 000...00

وهذه الحالة تستخدم للانتقال الآمن بين المعادلات لتجنب الحاجة إلى فحوصات الطوارئ لتجاوز السعة (overflow)

حالة NaN: فيها Exponent = 111...11 , Fraction ≠ 000...00

تستخدم هذه الحالة للإشارة إلى العمليات غير الغير صالحة والنتائج غير المعروفة مثل $\frac{0.0}{0.0}$, $\sqrt{-1}$

كما تستخدم للانتقال الآمن بين المعادلات

■ (sNaN) قيم NaN التنبيهية تتسبب فوراً في إحداث استثناء (exception) لعملية غير صالحة.

■ (qNaN) قيم NaN الهادئة تنتقل سلاطة عبر الحسابات دون تفعيل استثناء (exception) في ال Hardware



Interpretation of IEEE 754 Floating-Point Numbers

(b) binary 64 format

	Sign	Biased exponent	Fraction	Value
positive zero	0	0	0	0
negative zero	1	0	0	-0
plus infinity	0	all 1s	0	∞
Minus infinity	1	all 1s	0	$-\infty$
quiet NaN	0 or 1	all 1s	$\neq 0$; first bit = 1	qNaN
quiet NaN	0 or 1	all 1s	$\neq 0$; first bit = 0	sNaN
signaling NaN	0 or 1	all 1s	$\neq 0$; first bit = 0	sNaN
positive normal nonzero	0	$0 < e < 2047$	f	$2_{e-1023}(1.f)$
negative normal nonzero	1	$0 < e < 2047$	f	$-2_{e-1023}(1.f)$
positive subnormal	0	0	$f \neq 0$	$2_{e-1022}(0.f)$
negative subnormal	1	0	$f \neq 0$	$-2_{e-1022}(0.f)$

Interpretation of IEEE 754 Floating-Point Numbers

(a) binary 32 format

	Sign	Biased exponent	Fraction	Value
positive zero	0	0	0	0
negative zero	1	0	0	-0
plus infinity	0	all 1s	0	∞
Minus infinity	1	all 1s	0	$-\infty$
quiet NaN	0 or 1	all 1s	$\neq 0$; first bit = 1	qNaN
signaling NaN	0 or 1	all 1s	$\neq 0$; first bit = 0	sNaN
positive normal nonzero	0	$0 < e < 255$	f	$2_{e-127}(1.f)$
negative normal nonzero	1	$0 < e < 255$	f	$-2_{e-127}(1.f)$
positive subnormal	0	0	$f \neq 0$	$2_{e-126}(0.f)$
negative subnormal	1	0	$f \neq 0$	$-2_{e-126}(0.f)$

Interpretation of IEEE 754 Floating-Point Numbers

(c) binary 128 format

	Sign	Biased exponent	Fraction	Value
positive zero	0	0	0	0
negative zero	1	0	0	-0
plus infinity	0	all 1s	0	∞
minus infinity	1	all 1s	0	$-\infty$
quiet NaN	0 or 1	all 1s	$\neq 0$; first bit = 1	qNaN
signaling NaN	0 or 1	all 1s	$\neq 0$; first bit = 0	sNaN
positive normal nonzero	0	all 1s	f	$2_{e-16383}(1.f)$
negative normal nonzero	1	all 1s	f	$-2_{e-16383}(1.f)$
positive subnormal	0	0	$f \neq 0$	$2_{e-16383}(0.f)$
negative subnormal	1	0	$f \neq 0$	$-2_{e-16383}(0.f)$

جمع النقطة العائمة Floating Point Addition

لنفرض لدينا عدد مكون من 4 ارقام (Decimal) 9.999×10^1 و 1.610×10^{-1}

نتبع الخطوات التالية للقيام بعملية الجمع:

1. محاذاة الفواصل: نقوم بإزاحة الجزء المعياري الـ *significand* للعدد ذي الأس الأصغر (*Smaller Exponent*) نحو اليمين حتى يتطابق أسه مع أس الأكبر
 $9.999 \times 10^1 + 0.016 \times 10^1$
2. الجمع : جمع أو طرح الأجزاء المعيارية *significands* بناءً على إشارات المعاملات
 10.015×10^1
3. المعايرة (Normalization) : إزاحة الجزء المعياري لليسار أو اليمين لاستعادة القسم 1. الرئيسي طبعاً مع تعديل الأس وفقاً لذلك والتحقق من حدود تجاوز السعة أو التدفق السفلي
 1.0015×10^2 (*underflow, overflow*)
4. التقريب مع إعادة المعايرة إن كان ذلك ضرورياً: معالجة بتات الدقة الإضافية باستخدام عتاد التقريب وإعادة المعايرة إذا لزم الأمر 1.002×10^2

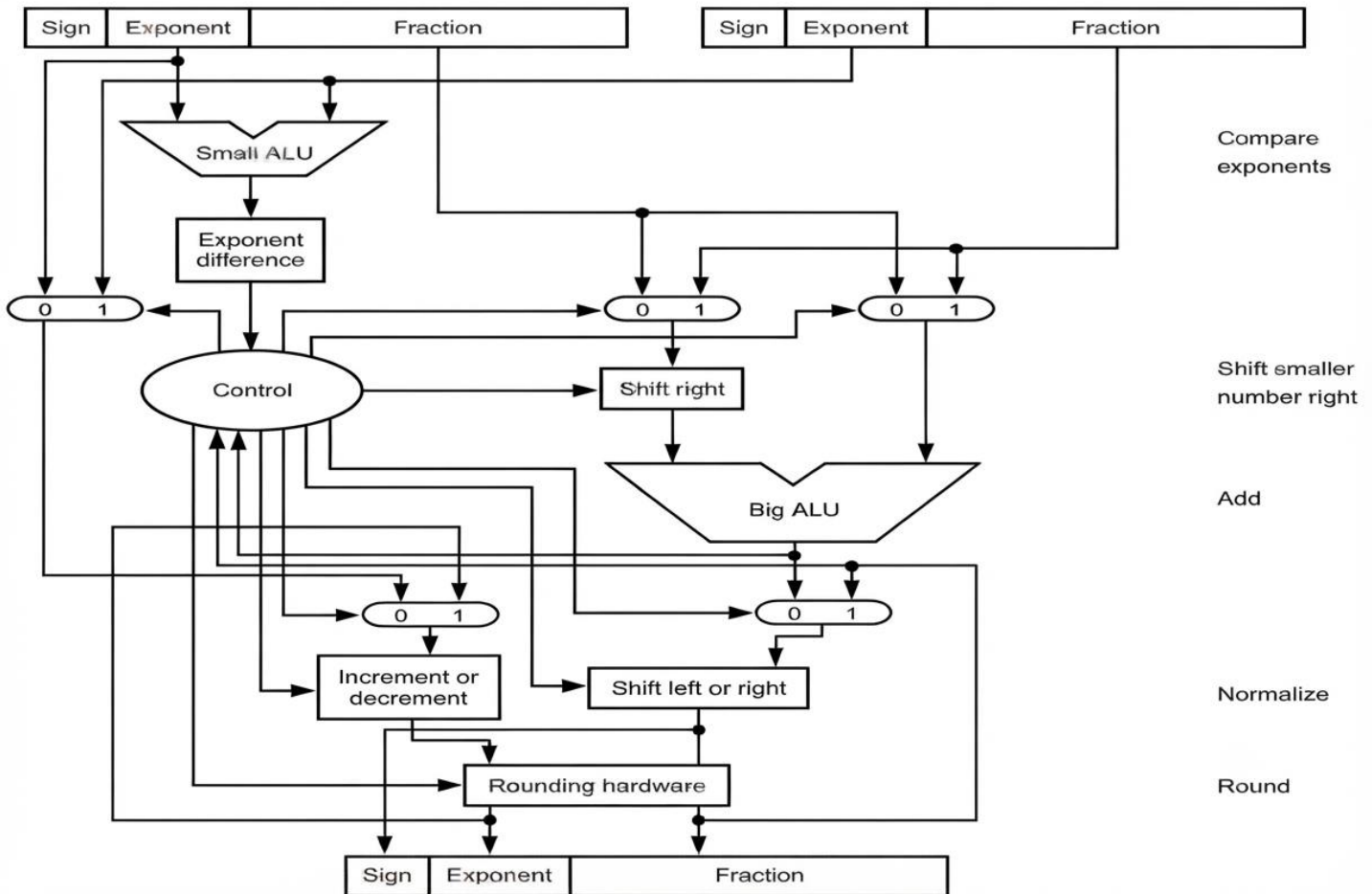
مثال: لدينا الرقم الثنائي المكون من 4 ارقام:

$$1.000_2 \times 2^{-1} + -1.110_2 \times 2^{-2}$$

باستخدام الخطوات السابقة سنقوم بإيجاد ناتج جمع الرقمين السابقين :

1. المحاذاة $1.000 \times 2^{-1} + (-0.111 \times 2^{-1})$
2. الجمع $1.000 \times 2^{-1} + (-0.111 \times 2^{-1}) = +0.001 \times 2^{-1}$
3. المعايرة مع التحقق من حالتي 1.000×2^{-4}
4. التقريب مع إعادة المعايرة إن لزم: $1.000 \times 2^{-4} = 0.0625$

Arithmetic Unit for Floating-Point Addition



ضرب النقطة العائمة Floating Point Multiplication

ليكن لدينا عدد مكون من 4 ارقام (Decimal) $1.110 \times 10^{10} \times 9.200 \times 10^{-5}$

سنقوم بأداء عدة خطوات للحصول على النتيجة المطلوبة:

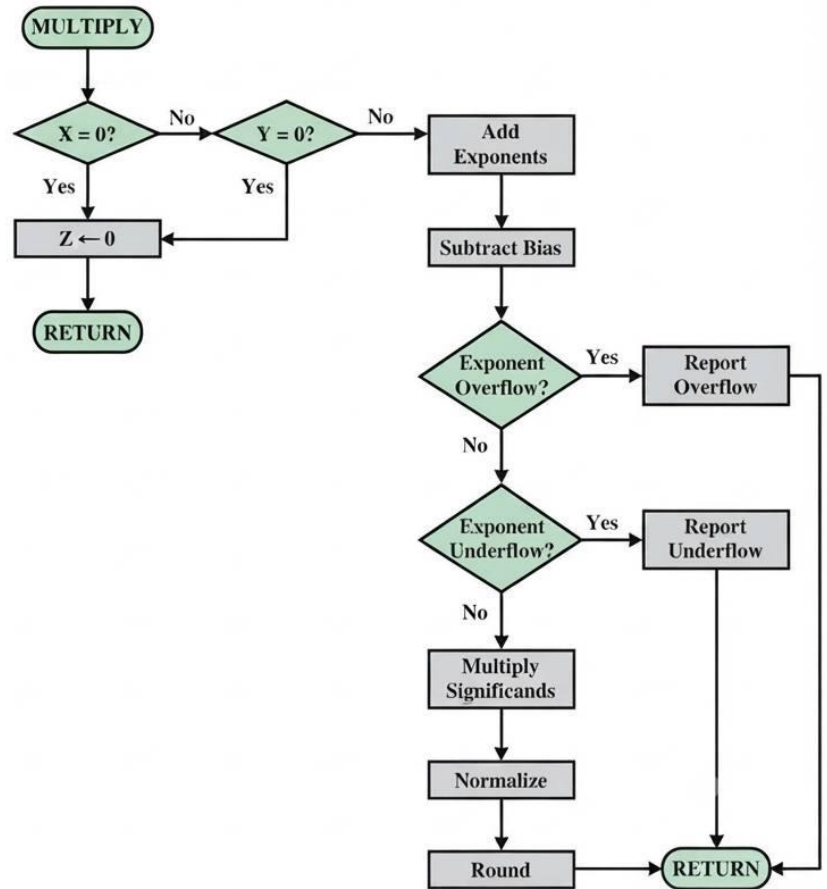
5. جمع الأس: (تذكر إذا كنت تستخدم Biased Exponent تذكر أن تطرح ال Bias الزائدة من المجموع الناتج) $10 + -5 = 5$
6. ضرب *signficands*: ضرب مصفوفات الكسور ببعضها البعض $1.110 \times 9.200 = 10.212$ أصبح لدينا 10.212×10^5
7. المعايرة مع التحقق من حالتي (underflow, overflow) 1.0212×10^6
8. التقريب مع إعادة المعايرة إن لزم: 1.021×10^6
9. تحديد الإشارة: من خلال ضرب الإشارات 1.021×10^6

الآن بعدما تعلمنا كيفية القيام بعملية الضرب لل Floating Point لنقوم بحل المثال التالي:

$$1.000_2 \times 2^{-1} + (-1.110_2) \times 2^{-2}$$

1. جمع الأس: $-1 - 2 = -3$ نقوم بإضافة ال Biased $(-1 + 127) + (-2 + 127) = -3 + 254 - 127 = -3 + 127 = 124$
2. ضرب *significands*: $1.000_2 \times 1.110_2 = 1.110_2$ $1.110_2 \times 2^{-3}$
3. المعايرة مع التحقق من حالتي (underflow, overflow): $1.110_2 \times 2^{-3} \rightarrow$
4. التقريب: $1.110_2 \times 2^{-3}$ (لا حاجة للتقريب) لا نعاير لأنه لا حاجة لذلك
5. تحديد الإشارة: $-1.110 \times 2^{-3} = -0.21875$

Floating-Point Multiplication



بضعة مفاهيم للأخذ بها للدقة Precision Considerations:

1. التقريب لأقرب : نقوم بتقريب النتيجة لأقرب رقم يمكن تمثيله أو كتابته
2. التقريب لل $+\infty$: نقوم بتقريب النتيجة نحو $+\infty$
3. التقريب لل $-\infty$: نقوم بتقريب النتيجة نحو $-\infty$
4. التقريب لل 0 : نقوم بتقريب النتيجة نحو 0



Precision Considerations

$x = 1.000\dots00 \times 2^1$ $-y = 0.111\dots11 \times 2^1$ $z = 0.000\dots01 \times 2^1$ $= 1.000\dots00 \times 2^{-22}$	$x = .100000 \times 16^1$ $-y = .0FFFFFF \times 16^1$ $z = .000001 \times 16^1$ $= .100000 \times 16^{-4}$
--	--

(a) Binary example, without guard bits

(c) Hexadecimal example, without guard bits

(b) Binary example, without guard bits

(d) Hexadecimal example, without guard bits

$x = 1.000\dots00 \ 0000 \times 2^1$ $-y = 0.111\dots11 \ 1000 \times 2^1$ $-y = 0.111\dots11 \ 1000 \times 2^1$ $z = 0.000\dots00 \ 1000 \times 2^1$ $= 1.000\dots00 \ 0000 \times 2^{-23}$	$x = .100000 \ 00 \times 16^1$ $-y = .0FFFFFF \ F0 \times 16^1$ $-y = .0FFFFFF \ F0 \times 16^1$ $z = .000000 \ 10 \times 16^1$ $= .100000 \ 00 \times 16^{-5}$
--	---

(b) Binary example, with guard bits

(d) Hexadecimal example, with guard bits

حساب الفترات Interval Arithmetic

توفر هذه المنهجية طريقة فعالة لمراقبة الأخطاء والتحكم فيها في حسابات Floating Point من خلال إنتاج قيمتين لكل نتيجة. تتوافق القيمتان مع النهايتين الدنيا والعليا لفترة تحوي على النتيجة الحقيقية. يشير عرض الفترة إلى دقة النتيجة:

1. إذا لم تستطع تمثيل النهايتين، فسيتم تقريبهما إلى الأسفل والأعلى بشكل متتالي.
 2. إذا كان النطاق بين الحدين العلوي والسفلي ضيقاً بما فيه الكفاية فسيتم الحصول على نتيجة دقيقة.
- * بعد التقريب إلى موجب ما لا نهاية وإلى سالب ما لا نهاية

الاقتطاع Truncation

الاقتطاع إما تقريب نحو الصفر، تجاهل بتات الزائدة، انحياز ثابت نحو الصفر في العملية وهو أبسط تقنية لكنه انحياز خطير لأنه يؤثر على كل عملية فيها بتات زائدة غير صفرية

معييار IEEE للحساب الثنائي لل Floating Point

اللانهاية: تعامل كحالة حدية للحساب الحقيقي، مع إعطاء قيم اللانهاية التفسير التالي:

$$-\infty > \text{كل رقم منتهي} > +\infty$$

$$5 \times (+\infty) = +\infty$$

$$5 + (+\infty) = +\infty$$

$$5 + (-\infty) = -\infty$$

$$5 - (+\infty) = -\infty$$

$$5 - (-\infty) = +\infty$$

$$(+\infty) + (+\infty) = +\infty$$

$$(-\infty) + (-\infty) = -\infty$$

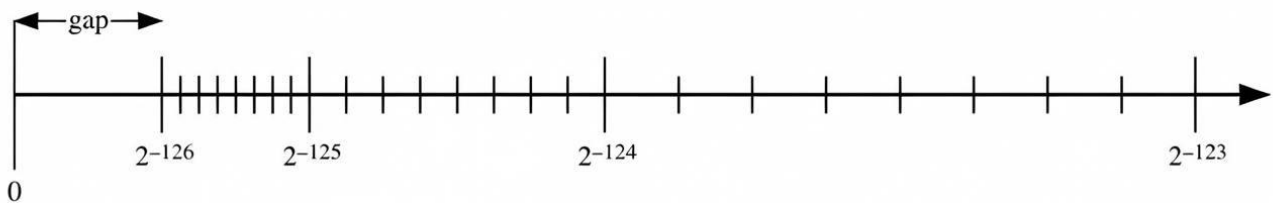
$$(-\infty) - (+\infty) = -\infty$$

$$(+\infty) - (-\infty) = +\infty$$

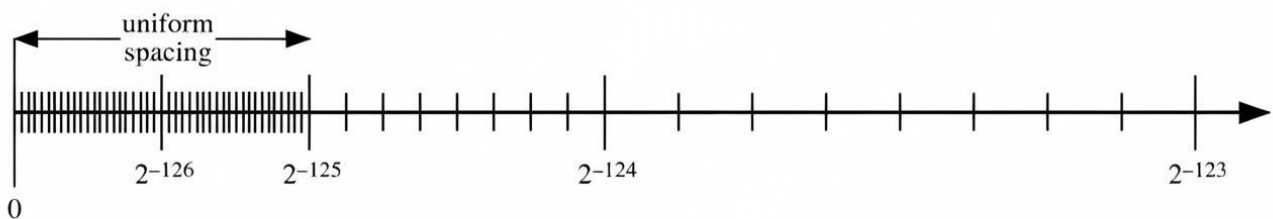
مثل:

تشير (NaN) المشيرة إلى استثناء عملية غير صالحة كلما ظهرت كمعامل.
(NaN) الهادئة تدخل تقريبا في جميع العمليات الحسابية دون الإشارة إلى أي استثناء.

Operation	Quiet NaN Produced by
Any	Any operation on a signaling NaN
Add or subtract	Magnitude subtraction of infinities: $(+\infty) + (-\infty)$ $(-\infty) + (+\infty)$ $(+\infty) - (+\infty)$ $(-\infty) - (-\infty)$ $(-\infty) - (-\infty)$
Multiply	$0 \times \infty$
Division	$\frac{0}{0}$ or $\frac{\infty}{\infty}$
Remainder	$x \text{ REM } 0$ or $\infty \text{ REM } y$
Square root	$\sqrt{x}$ where $x < 0$



(a) 32-bit format without subnormal numbers



(b) 32-bit format with subnormal numbers **with** subnormal numbers

■ مكونات جامع (FP Adder Hardware) FP

1. أكثر تعقيدا من جمع الأعداد الصحيحة.
2. القيام بالجمع بدورة ساعة كاملة سيأخذ العديد من الوقت وهذا أطول بكثير من مدة القيام بعمليات الأعداد الصحيحة. أيضاً دورات الساعة الأبطأ تقوم بإيقاف جميع العمليات
3. جمع ال Floating Point عادة يستغرق عدة دورات ساعة ولذلك يمكن استخدام (pipeline method) لتقسيم المهام لأجزاء أصغر يقام بحلها بنفس الوقت

■ مكونات FP Arithmetic

يشابه في التعقيد FP Adder لكنه يستخدم عوضاً عن المكونات الحسابية عادةً نقوم بعمليات جمع، طرح، ضرب، قسمة الجذور التربيعية، المقلوب. عادةً ما تستغرق عدة دورات ساعة ويمكن تنفيذها بشكل متسلسل.

حساب دقيق Accurate Arithmetic

يحدد معيار IEEE 754 تحكماً إضافياً في التقريب.

- بتات إضافية من الدقة (حماية، تقريب، تثبيت)
 - اختيار أوضاع التقريب.
 - يسمح للمبرمج بضبط السلوك العددي للحساب بدقة.
 - * لا تنفذ جميع وحدات الفاصلة العائمة جميع الخيارات ، حيث تستخدم معظم لغات البرمجة ومكتبات الفاصلة العائمة الإعدادات الافتراضية.
 - * المفاضلة بين تعقيد الأجهزة والأداة ومتطلبات السوق.
- (Multiple data)SIMD

مثال:

128-bit Adder

16 عملية جمع 8-bit

8 عمليات جمع 16-bit

4 عمليات جمع 32-bit

وأيضاً تسمى توازي على مستوى البيانات (data - level parallelism)

التوازي على مستوى الكلمات الفرعية Subword Parallelism

تطبيقات الصوت والرسومات قد تستفيد من إجراء عمليات متزامنة على متجهات قصيرة.

مثال:

وأيضاً تسمى توازي على مستوى البيانات (data- Level Parallelism) أو توازي المتجهين (vector Parallelism)

أو تعليمات واحدة (Single Instructions) ، بيانات متعددة

Associativity / التجميعية / المتوازية

قد تتداخل العمليات في البرنامج المتوازي بترتيبات غير متوقعة ولكن قد تفشل افتراضات التجميعية.

		$(x+y)+z$	$x+(y+z)$
x	-1.50E+38		-1.50E+38
y	1.50E+38	0.00E+00	
z	1.0	1.0	1.50E+38
		1.00E+00	0.00E+00

ولذلك نحن بحاجة إلى التحقق من صحة البرامج المتوازية في ظل درجات متفاوتة من التوازي.

من يهتم بدقة FP ؟ هي مهم هي مهمة للبرامج العلمية ولكن ماذا عن الاستخدام اليومي للمستهلك مثل : في حسابي البنكي خطأ بمقدار 0.0002 سنت

*لدينا مشكلة Intel Pentium FDIV

وهي عبارة عن مشكلة تقنية مشهورة في معالجات Pentium الأولى حيث كانت قسمة FP تعطي قيم غير دقيقة لبضعة أرقام وهذه وهذه مشكلة بالنسبة للسوق لأنه يتوقع أرقام دقيقة.

اقرأ تفاصيل أكثر من المرجع: Colwell The Pentium Chronicles

ملاحظات ختامية

1. البتات ليس لها معنى جوهري (وهذا يعتمد على التعليمات المطبقة) .
 2. تمثيلات الحاسوب للأعداد : له دقة ونطاق محدودان بما يتوجب علينا مراعاة ذلك في البرامج يمكن أن تحدث عمليات تجاوز (overflow) أو نقص في النطاق (underflow).
 3. ISA تدعم الحسابات:
 - للأرقام الأعداد الصحيحة (unsigned) و (signed)
 - تقريب الفاصل العائم للأعداد الحقيقية.
 4. تعليمات أساسية: 54 منها أكثر استخداماً
 - 100% من SPECINT
 - 97% من SPECFP
- *باقي التعليمات أقل استخداماً.

